DP Computer Science: A 3.3.2 Constructing Queries in SQL

---

1. Core Concepts & Learning Objectives

Learning Intentions

By the end of this lesson, students will be able to:

- Explain the purpose of using **JOIN** clauses to combine data from two tables in SQL
- Construct `SELECT` queries that include different types of **JOIN** clauses (`INNER JOIN`, `LEFT JOIN`, `RIGHT JOIN`)
- Apply **relational operators** (`=`, `>`, `<`, `>=`, `<=`, `!=`, `<>`) within the `WHERE` clause to filter data across two joined tables
- Utilise **logical operators** (`AND`, `OR`, `NOT`) to combine multiple filtering conditions
- Implement **pattern matching** using the `LIKE` operator with the `%` wildcard
- Order results using the `ORDER BY` clause with `ASC` and `DESC`
- Use `BETWEEN` to filter data within a specified range
- Apply `DISTINCT` to retrieve unique records

Key Vocabulary

Term	Definition
JOIN	A clause used to combine rows from two or more tables based on a related column
INNER JOIN	Returns only rows that have matching values in both tables
LEFT JOIN	Returns all rows from the left table, and matching rows from the right table
RIGHT JOIN	Returns all rows from the right table, and matching rows from the left table

Term	Definition
ON clause	Specifies the condition for joining two tables (the matching columns)
Relational operator	Compares two values (`=` , `>` , `<` , `>=` , `<=` , `!=`)
Logical operator	Combines multiple conditions (`AND` , `OR` , `NOT`)
LIKE	Used for pattern matching in text searches
Wildcard	A symbol used in pattern matching (`%` matches any sequence of characters)
ORDER BY	Sorts query results in ascending (`ASC`) or descending (`DESC`) order
DISTINCT	Returns only unique (non-duplicate) values
BETWEEN	Filters values within a specified range

Relevant ATL Skills

Skill	Description
Thinking	Critical thinking (analysing relationships); problem-solving (constructing queries)
Communication	Using precise language to describe queries; interpreting results
Research	Locating and using information about SQL syntax
Self-Management	Organising and structuring SQL queries logically

---

2. Why Query Two Tables?

The Need for JOINS

> **In a relational database, data is often spread across multiple tables. To answer questions that involve data from more than one table, you need to combine (join) them.**

Real-World Examples

Scenario	Tables Involved	Information Needed
E-Commerce	Customers + Orders	Which customers have placed orders?
School System	Students + Courses	Which students are enrolled in which courses?
Library	Books + Authors	Which books were written by a specific author?
Company	Employees + Departments	Which employees work in which department?

A Simple Example: Customers and Orders

```
text

┌──────────────────────────────────────────────────────────────────┐
│               WHY JOIN TABLES?                                   │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│    ┌──────────────────┐          ┌──────────────────┐            │
│    │   Customers      │          │    Orders        │            │
│    ├──────────────────┤          ├──────────────────┤            │
│    │ CustomerID (PK) │◄─────────┤ CustomerID (FK) │            │
│    │ FirstName        │          │ OrderID          │            │
│    │ LastName         │          │ OrderDate        │            │
│    │ Email            │          │ Total            │            │
│    └──────────────────┘          └──────────────────┘            │
│                                                                  │
│    To find: "Which customers have placed orders over $100?"      │
│                                                                  │
│    We need data from BOTH tables:                                │
│    • Customers table: first name, last name                      │
```

```
│    • Orders table: total                                    │
│                                                             │
│    Without JOIN, we would need two separate queries!        │
│                                                             │
└─────────────────────────────────────────────────────────────┘
```

---

3. Introducing the JOIN Clause

Basic JOIN Syntax

```sql
SELECT column1, column2, ...
FROM table1
JOIN table2
ON table1.column = table2.column
WHERE condition;
```

The ON Condition

> The `ON` clause specifies which columns in the two tables match. It tells the database how to link the tables together.

Aspect	Description
Purpose	Defines the relationship between the tables
Usually	Primary Key (PK) from one table equals Foreign Key (FK) from the other
Example	`ON Customers.CustomerID = Orders.CustomerID`

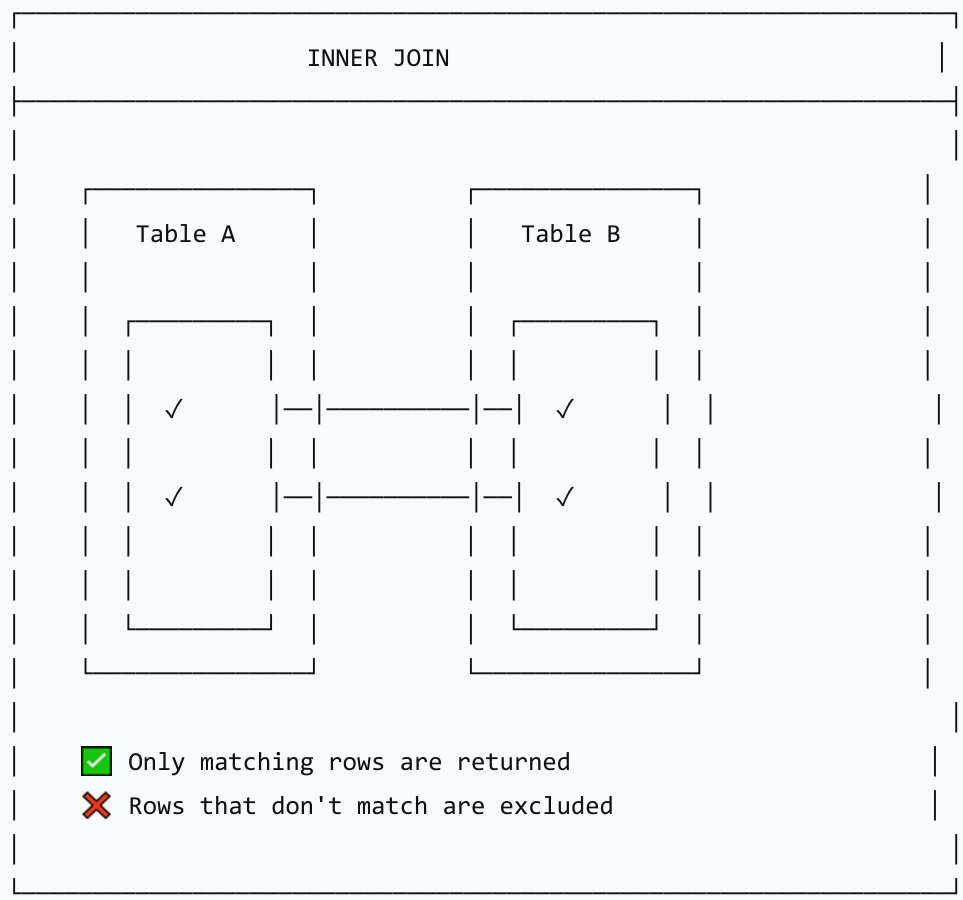
---

4. Types of JOINs

INNER JOIN

> Returns only rows that have matching values in BOTH tables.

text



INNER JOIN Example

sql

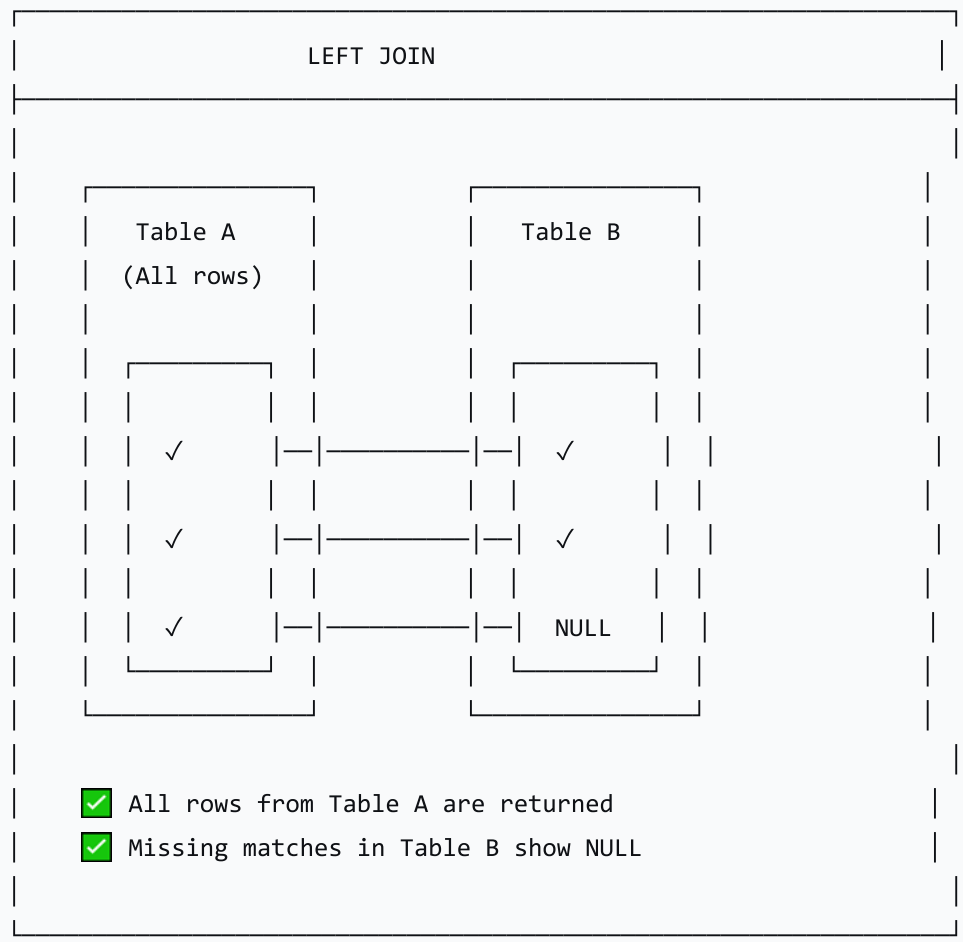
```sql
SELECT Customers.FirstName, Customers.LastName, Orders.OrderDate, Orders.Total
FROM Customers
INNER JOIN Orders
ON Customers.CustomerID = Orders.CustomerID;
```

Result: Shows customers who have placed orders, and the details of their orders.

LEFT JOIN

> **Returns ALL rows from the left table, and matching rows from the right table. If no match, NULL values are returned for the right table's columns.**

text



LEFT JOIN Example

sql

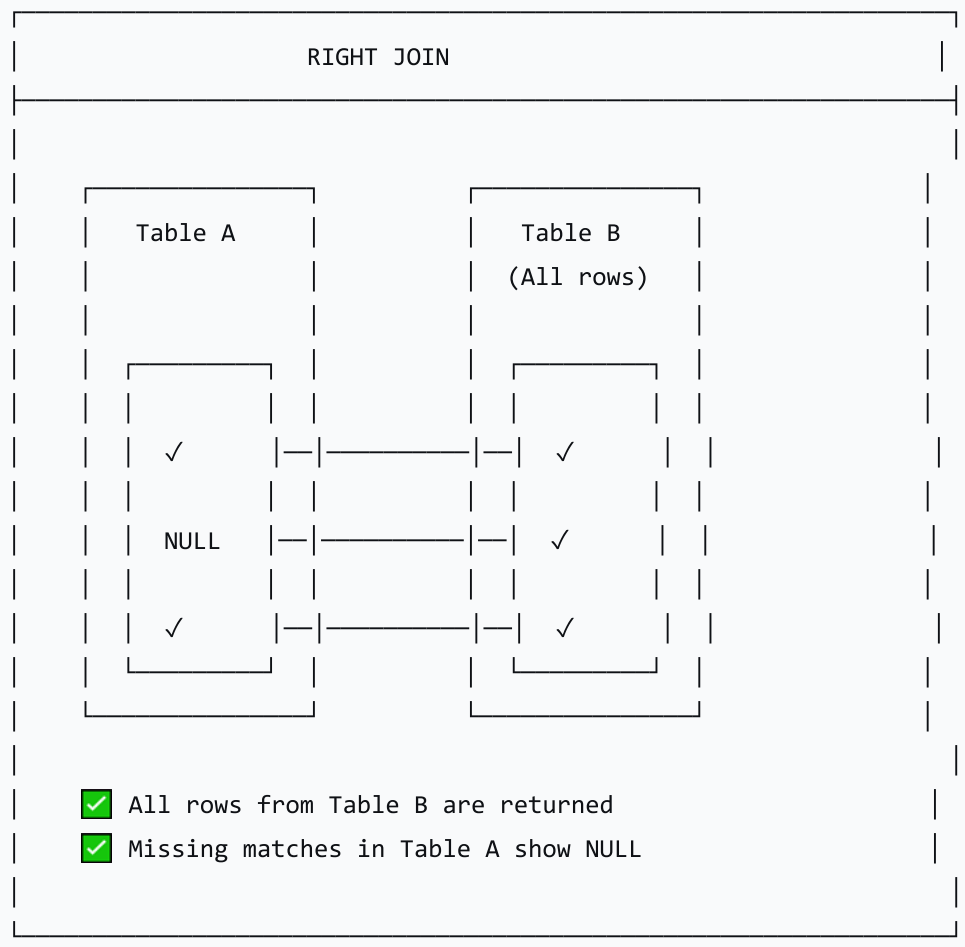
```sql
SELECT Customers.FirstName, Customers.LastName, Orders.OrderDate, Orders.Total
FROM Customers
LEFT JOIN Orders
ON Customers.CustomerID = Orders.CustomerID;
```

Result: Shows ALL customers, even those who have not placed any orders (they will have NULL in the order columns).

RIGHT JOIN

> Returns ALL rows from the right table, and matching rows from the left table. If no match, NULL values are returned for the left table's columns.

text



RIGHT JOIN Example

sql

```sql
SELECT Customers.FirstName, Customers.LastName, Orders.OrderDate, Orders.Total
FROM Customers
RIGHT JOIN Orders
ON Customers.CustomerID = Orders.CustomerID;
```

JOIN Comparison Summary

JOIN Type	Description	Result
INNER JOIN	Matches in both tables	Returns only matched rows
LEFT JOIN	All from left; matches from right	Returns all left rows; NULL where no match
RIGHT JOIN	All from right; matches from left	Returns all right rows; NULL where no match

---

5. Filtering Data with WHERE

Using Relational Operators

Operator	Description	Example
`=`	Equal to	`WHERE Total = 100`
`>`	Greater than	`WHERE Total > 100`
`<`	Less than	`WHERE Total < 50`
`>=`	Greater than or equal	`WHERE Total >= 100`
`<=`	Less than or equal	`WHERE Total <= 50`
`!=`	Not equal	`WHERE Status != 'Cancelled'`
`<>`	Not equal	`WHERE Status <> 'Cancelled'`

Using Logical Operators

Operator	Description	Example
AND	Both conditions must be true	`WHERE Total > 100 AND CustomerID = 1`
OR	At least one condition must be true	`WHERE Total > 100 OR Total < 50`
NOT	Negates a condition	`WHERE NOT Status = 'Cancelled'`

Filtering Across Joined Tables

```sql
SELECT Customers.FirstName, Customers.LastName, Orders.OrderDate, Orders.Total
FROM Customers
INNER JOIN Orders
ON Customers.CustomerID = Orders.CustomerID
WHERE Orders.Total > 100
AND Orders.OrderDate >= '2026-01-01';
```

---

6. Pattern Matching with LIKE

The LIKE Operator

> The `LIKE` operator is used for pattern matching in text searches. The `%` wildcard matches any sequence of characters.

Pattern	Matches	Example
`'a%'`	Starts with 'a'	'apple', 'ant'
`'%a'`	Ends with 'a'	'banana', 'soda'
`'%a%'`	Contains 'a'	'banana', 'apple'
`'a___'`	Starts with 'a' and has 3 more characters	'apple' (5 chars)

LIKE Example with JOIN

```sql
SELECT Customers.FirstName, Customers.LastName, Customers.Email
FROM Customers
INNER JOIN Orders
ON Customers.CustomerID = Orders.CustomerID
WHERE Customers.LastName LIKE 'S%';
```

Result: Customers whose last name starts with 'S' who have placed orders.

---

7. Ordering Data with ORDER BY

The ORDER BY Clause

> The `ORDER BY` clause sorts query results. Use `ASC` for ascending (default) or `DESC` for descending.

Keyword	Description	Example
ASC	Ascending (A to Z, 1 to 10)	`ORDER BY LastName ASC`
DESC	Descending (Z to A, 10 to 1)	`ORDER BY Total DESC`

ORDER BY Example with JOIN

```sql
SELECT Customers.FirstName, Customers.LastName, Orders.Total
FROM Customers
INNER JOIN Orders
ON Customers.CustomerID = Orders.CustomerID
ORDER BY Orders.Total DESC;
```

Result: Customers and their order totals, sorted from highest to lowest.

---

8. Other Useful Commands

DISTINCT

> `DISTINCT` returns only unique (non-duplicate) values. Useful for finding distinct customers who have placed orders.

```sql
SELECT DISTINCT Customers.CustomerID, Customers.FirstName, Customers.LastName
FROM Customers
INNER JOIN Orders
ON Customers.CustomerID = Orders.CustomerID;
```

BETWEEN

> `BETWEEN` filters values within a specified range (inclusive).

```sql
SELECT Customers.FirstName, Customers.LastName, Orders.Total
FROM Customers
INNER JOIN Orders
ON Customers.CustomerID = Orders.CustomerID
WHERE Orders.Total BETWEEN 50 AND 200;
```

---

9. Complete Query Example

Problem: "Find all customers who have placed orders over $100 in 2026, sorted by total amount"

```sql
SELECT DISTINCT
    Customers.FirstName,
    Customers.LastName,
    Orders.OrderID,
```

```
    Orders.OrderDate,
    Orders.Total
FROM Customers
INNER JOIN Orders
    ON Customers.CustomerID = Orders.CustomerID
WHERE Orders.Total > 100
    AND Orders.OrderDate >= '2026-01-01'
    AND Orders.OrderDate <= '2026-12-31'
ORDER BY Orders.Total DESC;
```

Breakdown of Components

Component	Description
`SELECT DISTINCT`	Retrieves unique rows (avoids duplicates)
`FROM Customers INNER JOIN Orders`	Joins the two tables based on CustomerID
`ON Customers.CustomerID = Orders.CustomerID`	Defines the matching condition
`WHERE Orders.Total > 100`	Filters for orders over $100
`AND Orders.OrderDate BETWEEN ...`	Filters for orders in 2026
`ORDER BY Orders.Total DESC`	Sorts by total amount (highest first)

---

10. Lesson Sequence

Lesson Plan (60 minutes)

Phase	Time	Activity
Introduction	5 min	Lesson objectives; real-world examples of why we query two tables

Phase	Time	Activity
JOIN Basics	10 min	Purpose of JOIN; basic syntax; ON condition
Types of JOINs	10 min	INNER JOIN, LEFT JOIN, RIGHT JOIN; visual representations; SQL examples
Filtering with WHERE	10 min	Relational and logical operators; filtering across joined tables
Pattern Matching & Ordering	10 min	LIKE with % wildcard; ORDER BY with ASC and DESC
Other Commands	5 min	DISTINCT, BETWEEN
Summary & Activity	10 min	Recap; introduce worksheet activity

---

Activity: Worksheet Practice

Task	Description
Task 1	Write an INNER JOIN query to display customer names and their order totals
Task 2	Write a LEFT JOIN query to display all customers, including those with no orders
Task 3	Add a WHERE clause to filter orders over a specific amount
Task 4	Use LIKE to find customers with a specific pattern in their name
Task 5	Use ORDER BY to sort results by date or total amount

---

11. Assessment Activities

Assessment Type	Description
Class Participation	Observe engagement and contributions
Worksheet	Evaluate ability to construct queries with JOINS
Exit Ticket	Gauge understanding and identify areas needing clarification

Sample Exit Ticket Questions:

1. What is the purpose of a JOIN in SQL?
2. What is the difference between INNER JOIN and LEFT JOIN?
3. Write a query to find all customers and their orders, using INNER JOIN.
4. How would you filter for orders over $100?

---

12. Differentiation

Level	Strategy
Support	Provide scaffolded query structures; work through first few examples together; hints on JOIN type
Challenge	Challenge questions in worksheet; encourage students to create their own query scenarios

---

13. Resources

- **Presentation** (Slide Deck)
- **eBook** (A3.3.2 with guided examples)
- **Worksheet:** JOIN queries with multiple tasks
- **Worksheet Answers**
- **DB Browser for SQLite** (software for hands-on practice)

---

14. Summary: Key Takeaways

Concept	Key Point
JOIN	Combines rows from two or more tables
INNER JOIN	Returns only matching rows
LEFT JOIN	Returns all rows from the left table
RIGHT JOIN	Returns all rows from the right table
ON	Specifies the matching condition
WHERE	Filters results (relational/logical operators)
LIKE	Pattern matching in text (`%` wildcard)
ORDER BY	Sorts results (`ASC` , `DESC`)
DISTINCT	Returns unique values
BETWEEN	Filters within a range

> *"JOINs are the key to unlocking the power of relational databases—they allow you to combine data from multiple tables to answer complex questions."*